Real-Time Lip Movement Detection and Command Execution in Robotics: A MediaPipe and LangChain Integrated Approach

Sebastian Joel Rivett Macias, *MCC - Student , INAOE,*

***Abstract*—The main purpose of the project focuses on leveraging modern technology to enable human-robot or human-computer interaction. Specifically, the project utilizes MediaPipe to detect lip movements (comparing information of mean difference lips gap in the actual frame vs the last frame). This mechanism triggers the Whisper system to convert spoken instructions into text. Subsequently, a large language model (LLM) powered by LangChain and Ollama performing the process of the text to interpret and execute commands within a simulated drone environment in Gazebo on Ubuntu 22.04 LTS.**

***Index Terms*—Technology, Human-robot interaction, Human-computer interaction, MediaPipe, Lip movements, Whisper system, Spoken instructions, Text conversion, Large language model (LLM), LangChain, Ollama, Command interpretation, Drone simulation, Gazebo, Ubuntu 22.04 LTS**

I. Introduction

A. Whisper

Whisper is an automatic speech recognition (ASR) system developed using 680,000 hours of diverse, multilingual data from the web. It demonstrates enhanced robustness against accents, background noise, and technical language, and supports transcription in multiple languages and translation into English. The models and inference code are open-sourced to facilitate further research and application development in robust speech processing. [1]

Training Data: [1]

- **English-Language Data:** 65% of the training data, or 438,000 hours, consists of English audio and corresponding English transcripts.
- **Non-English Audio with English Transcripts:** Approximately 18%, or 126,000 hours, of the data involves non-English audio paired with English transcripts.
- **Non-English Audio and Transcripts:** 17%, or 117,000 hours, consists of non-English audio with transcripts in the same languages, covering 98 different languages.

Size	Parameters	Multilingual Model	Required VRAM
tiny	39 M	tiny	∼1 GB
base	74 M	base	∼1 GB
small	244 M	small	∼2 GB
medium	769 M	medium	∼5 GB
large	1550 M	large	∼10 GB

TABLE I
MODEL SPECIFICATIONS ABOUT VARIANTS OF WHISPER [2].

Limitations: The actual models of whisper (V1, V2, V3) sometimes generate text not present in the audio input, a phenomenon termed as ”hallucination.” This is hypothesized to occur because the models, leveraging their general knowledge of language, attempt to predict subsequent words while also transcribing the audio. (To avoid that it’s necessary to consider multiples executions of processing one same file with whisper, that can allow to be the exact as possible in the transcription but with the price of not be in actual time). [1]

Use cases of Whisper: [3]

- Transcription of audio files.
- Language Learning Tools
- Just In Time Subtitles (Used on Twich, Davinci Resolve, Youtube, etc.)
- Translation of text to another language.

B. Mediapipe

MediaPipe is a framework developed by Google for building machine learning pipelines to process time-series data like video and audio. It is versatile, supporting multiple platforms including desktop/server, Android, iOS, and embedded devices such as Raspberry Pi and Jetson Nano. Initially created for real-time video and audio analysis on YouTube, MediaPipe has since been integrated into numerous Google products like NestCam, Google Lens, Google Photos, and Gmail, among others. [8]

The Face Landmarker uses a series of models to predict face landmarks. The first model detects faces, a second model locates landmarks on the detected faces, and a third model uses those landmarks to identify facial features and expressions. [5]

Models presents on Mediapipe: [5]

- **Face detection model:** Detects the presence of faces with a few key facial landmarks.
- **Face mesh model:** Adds a complete mapping of the face. The model outputs an estimate of 478 3-dimensional face landmarks.
- **Blendshape prediction model:** Receives output from the face mesh model predicts 52 blendshape scores, which are coefficients representing facial different expressions.

C. Langchain

Framework designed to enhance the development and deployment of applications powered by large language

models (LLMs), which is a critical aspect of modern computational technologies, particularly in the field of robotics. It provides a resource that enables users to control LLMs within a Python program installed in a PC environment, utilizing them as classifiers, predictors, selectors (and a way to almost eradicate SQL because depending on the select model, can be used as intermediary or even without an SQL Framework to make queries) , and more ways to use the LLM's. [12]

Use of LangChain: Used to streamline the development of NLP applications by providing a big infrastructure for integrating various LLM's at the same time, and managing the flow of operations through pipelines. These pipelines, as previously mentioned, organize the process from input handling to final output delivery, ensuring efficient data transformation and decision-making. [9] [11] [12]

Integration with Multiple LLMs: Supports integration with a variety of LLMs. For example, LangChain can be configured to work with models like OpenAI's GPT-3 for general-purpose tasks, Anthropic's Claude for nuanced text generation, or other specialized models depending on the project's needs. [13] [14]

Key Features and Components:

- **Modular Design:** LangChain is built with modularity in mind, which allows for flexible assembly of components like retrieval systems, processing chains, and output modules. This design facilitates easy customization and scalability.
- **LangChain-Core:** Provides foundational elements and abstractions, making it easier to build and modify language processing chains.
- **LangChain-Community:** Supports third-party integrations and enhances connectivity to various LLM platforms.
- **LangGraph:** A tool for constructing robust applications by modeling processes as a network of nodes and edges.
- **LangSmith:** Offers tools for debugging, testing, and monitoring LangChain applications, ensuring high reliability and optimal performance.
- **LangServe:** Facilitates the deployment of LangChain applications as REST APIs, making them accessible over networks and easy to integrate with other systems.

D. Ollama

Ollama is a part of the LangChain framework, which integrates with various large language models (LLMs) to enhance the capabilities of applications that rely on natural language processing. [16]

Ollama acts as an interface or a wrapper that facilitates interaction with different LLMs. It streamlines the process of sending queries to and receiving responses from these models. By abstracting the complexities involved in directly interacting with LLM APIs, Ollama allows developers to focus more on the application logic rather than the intricacies of model communication. [16] [17]

Model	Parameters	Size
Llama 3	8B	4.7GB
Llama 3	70B	40GB
Phi-3	3.8B	2.3GB
Mistral	7B	4.1GB
Mixtral	141B	79.1GB

TABLE II
MODEL SPECIFICATIONS EXAMPLES. [19]

Llama3: LLaMA (LLaMA stands for "LLaMA: Low-Latency Matrix Algebra"), developed by Meta AI, is a family of transformer-based LLM's designed for efficiency and performance across a range of computing environments. LLaMA 3 it's the latest model developed. [22] [23]

Comparison between LLaMA 2 and LLaMA 3: [23]

- **Model Size and Capacity:** LLaMA 2 and its predecessors typically came in various sizes, each suited to different computational and application needs. LLaMA 3 continues this trend but generally offers more refined architectures that improve on the efficiency and effectiveness of the model, processing more data more quickly and accurately.
- **Performance Improvements:** Each iteration of LLaMA models aims to reduce latency and improve accuracy in understanding natural language. LLaMA 3 likely introduces enhancements that minimize response times and improve the quality of the model outputs, crucial for real-time robotics applications.
- **Technological Advances:** LLaMA 3 incorporate newer techniques in AI research, such as advanced training algorithms, better handling of biases, and greater generalization capabilities across different languages and tasks.

II. REVIEW OF THE STATE OF THE ART

A. A Whisper ROS Wrapper to Enable Automatic Speech Recognition in Embedded Systems

Technical Challenges and Solutions: Integrating an Automatic Speech Recognition, (ASR) system like Whisper into ROS, while technically straightforward due to ROS's flexibility and multi-language support. Whisper.cpp has been successfully integrated into ROS using popular embedded platforms like Raspberry Pi, which are portable, energy-efficient, and discretely sized for easy inclusion in commercial robotic platforms. [24]

Integration of ASR in ROS for Conversational Robots: For conversational robots that simultaneously manage sensor data, actuator control, decision-making, and user interaction, the integration of an (ASR) system into the (ROS) is performed on-demand. ROS Services and Actions, serving as callback blocking functions activated by specific signals, facilitate this integration. The implemented ROS service supports real-time speech recognition by connecting to all network nodes and capturing audio through microphones, thereby improving robot responsiveness in interactive environments. [24]

Modes of Operation With ROS: The ROS service operates in three modes: The first mode streams and transcribes audio continuously for a set period (T seconds). The second mode listens passively, activating transcription upon detection of a specific passphrase, and then transcribes a pre-set number of words. The third mode transcribes and matches keywords from a vocabulary, issuing commands corresponding to each recognized keyword. [24]

Results - Testing the Inference Process: An 11 second audio sample was transcribed to measure the performance of the inference process using 1500 context tokens. The measurements included load time, encoder time, and total inference time using the ROS Service on Raspberry hardware. The model load time was approximately 2000ms, encoding took 7600ms, and the total inference process lasted about 10 seconds. Reducing the context tokens to 768 and 512 resulted in the encoder running two and three times faster, respectively. This adjustment allows the ROS service to transcribe 3 to 7 second audio samples with almost real time performance. [24]

B. Web Scraping for Ovarian Cancer Detection: Utilizing Open-Source WhisperAI for Identifying Relevant Terminology and Improving Early Diagnosis

Effectiveness of ASR in Detecting Chemical Entities for Ovarian Cancer: Ovarian cancer requires early detection for effective treatment, and the proposed Automatic Speech Recognition (ASR) system leverages deep learning models to recognize complex speech patterns and differentiate between various chemical terms pertinent to ovarian cancer. Recognized chemical entities are employed for web content search and retrieval. The study underscores the potential of ASR technology in early detection and precise identification of chemical entities linked to ovarian cancer, and its utility in enhancing web-based information retrieval. [25]

Model Leveraging Whisper's Capabilities:

- **Speech Acquisition:** High-quality speech acquisition is ensured through the selection and optimal placement of a high-quality directional microphone. This microphone is positioned to capture sound directly from the participant's mouth, minimizing ambient noise and ensuring sound waves are captured with high fidelity.
- **Speech Preprocessing:** Involves the transforming the raw audio waveform into a spectrogram, which displays the frequency content over time. This spectrogram feeds into an encoder network—either a convolutional neural network (CNN) or a bidirectional recurrent neural network (BiRNN), which extracts critical temporal and spectral features, outputting a bottleneck vector summarizing these features.
- **Decoder Network Functionality:** The bottleneck vector is input into a decoder network (typically an RNN), which reconstructs a spectrogram from the vector. This spectrogram is then transformed back into an audio waveform using a vocoder.
- **Text Normalization and Keyword Extraction:** After converting speech to text, the text is normalized by removing non-textual elements, converting to lowercase, and tokenizing into words. Keywords are then extracted using either a manually compiled vocabulary or automatic tools like PubChem, and matched against a chemical terms dictionary.
- **PubMed Web Scraping:** The extracted keywords are used to search the PubMed database through web scraping, retrieving details like article titles, authors, abstracts, and publication dates to support further research and applications.

The proposed model processes voice queries to extract terms related to ovarian cancer, such as "cisplatin," "ovarian," and "cancer." It combines these terms into a single string and searches PubMed and Google to fetch research papers. The gathered data, including titles, abstracts, publication years, authors, and journals, is then saved into a .csv file, optimizing the research process by reducing manual search efforts and potential errors. [25]

C. Automating Customer Service using LangChain

Novel Open-Source Approach to Building LLM Chatbots: A new open-source framework is proposed for building LLM chatbots using website content. It is designed to be robust, accommodating various types of datasets found on web-pages or linked webs. This framework is intended to enhance FAQ systems with a more interactive and user-friendly interface. A comparative analysis of various models was conducted to evaluate their performance on the provided data against the expected response from the LLM. [26]

LLM Chatbot Development Components:

- **Data Collection:** Data was sourced using Beautiful-Soup web scraping from an organization's homepage, capturing diverse content like FAQs, manuals, forums, and chat logs to inform the LLM's context.
- **Embeddings:** Semantic and contextual information crucial for LLMs were captured using the "hkunlp/instructor-large" model from Hugging-Face Instruct Embeddings, stored via the FAISS library for efficient search.
- **Language Model Selection:** Google's Flan T5 XXL was chosen for its ability to effectively maintain conversation context and support dynamic interactions, enhancing the chatbot's user engagement.
- **Integration with Customer Service Platforms:** A chat window enabled by the Gradio API framework was implemented for user interaction, demonstrating the chatbot's capabilities on a website.

Innovative Approach in Customer Service Using LangChain: The open-source Large Language Model (LLM) architecture (in company with the framework Langchain) revolutionizes the relationship between customers and companies, enhancing customer experience (CX). It allows for efficient, personalized, and responsive support, which contributes to customer retention, increases customer value, and improves brand image. [26]

TABLE III
QUERY RESPONSES AND PERFORMANCE METRICS OF GOOGLE'S FLAN-T5-XXL PERFORMANCE

Sr. No.	Query/Prompt	Answer	Performance
1	What is BVM?	Birla Vishvakarma Mahavidyalaya	4 / 5
2	Where is it?	Vallabh Vidyanagar, Gujarat India	3 / 5
3	What is IEEE BVM?	Institute of Electrical and Electronics Engineers Student Branch of BVM	4 / 5
4	What is TRS BVM?	BVM Student Chapter associated with Robotics Society India	5 / 5
5	Name the annual newsletter?	Vishvakarma Magazine and Newsletter	3 / 5
6	Tell me something about ICWSTCSC	The International Conference for Women in Science and Technology Creating Sustainable Career is the 3rd International Conference happening in hybrid mode.	5 / 5

D. A Family of Natural Language Interfaces for Databases based on ChatGPT and LangChain

Functionality of ChatGPT in Query Translation and Keyword Extraction: The OpenAI API's Chat Completions API takes a list of messages, categorized by roles like "system," "user," or "assistant," to generate responses. System messages dictate the assistant's behavior, user messages pose inquiries, and assistant messages provide ChatGPT's responses. ChatGPT can also convert natural language questions into SQL queries and extract keywords. For SQL translation, users can either input minimal context for the model to utilize its built-in knowledge or add a detailed system message with specific metadata to refine the translation process. This is called Natural Language (NL) interfaces for databases (NLIDBs). [27]

Using Frameworks for Developing LLM-Powered Applications: LangChain frameworks develop applications using large language models (LLMs) by employing "Chains" that sequence multiple calls to LLMs or other utilities. The SQLDatabaseChain, for instance, analyzes a database's schema, tables, and joins, automatically or manually enriching LLMs with relevant context. This enables ChatGPT to formulate SQL queries from natural language questions, which are then executed and returned by LangChain. In other words in some SQL enviroments [27]

The Chat Completions of the modified version of CHATGPT, enables the description of function calls for responses when the information from the LLM is insufficient or incomplete, facilitating the creation of a natural language front-end for database keyword searches. This allow us to omit the necessity of learning SQL and using only prompts to generate queries of tables presents on SQL enviroments.

E. Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast

Chain component present in Lanchaing: The fundamental component of LangChain is the chain, which typically integrates an LLM with a prompt. Multiple such building blocks can be combined to perform a series of operations on text or other data. A simple chain processes a single input prompt to generate an output. Chains can be executed sequentially, where the output of one serves as the input for the next. These chains can be linked using the Simple Sequential Chain class for processes involving a single input and output. [28]

Multiple Chains: LangChain also offers a SequentialChain class designed for scenarios with multiple inputs but a single output. A common use case involveing multiple chains (each one specialized for different types of inputs) and a router chain determines which subchain an input should be passed to based on the nature of the input. [28]

Router of Multiple Chains: Each chain may employ either different or the same LLM, primarily distinguished by their prompts. These prompts, combined with user input, describe the role of the chain and the expected output, ensuring the chain functions as intended. [28]

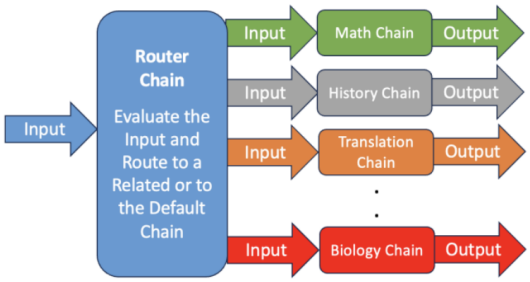

Fig. 1. Router Chain that affects the input of a chain that can answer the input the best. [28]

Chat with our Documents One common and complex application built using an LLM is a system designed to answer questions about documents such as PDFs, webpages, or CSV files. This system uses an LLM to help users gain a deeper understanding of document contents and access needed information. LLMs can process data beyond their original training, enabling a variety of use cases. LangChain simplifies the processes of loading, transforming, storing, and querying data. It includes several classes to support these functions:

- **Document Loaders:** Import documents from various sources such as CSV, PDF, HTML, JSON, Excel, and others.
- **Document Transformers:** Break documents into smaller parts for LLM processing.
- **Text Embedding Models:** Vector Stores: Store and manage data based on these embeddings.
- **Retrievers:** Query data using embedding similarity.

Agents: Agents facilitate flexible sequences of calls to LLMs and other tools in applications, adapting to user input. There are two types: action agents, which adjust their next action based on prior outputs, and plan-and-execute agents, which map out and follow a fixed sequence of actions without modification. Action Agents process user inputs through selected tools, continuously adapting until a direct response is possible, managed by agent executors. Plan and Execute Agents pre-plan a complete sequence based on the initial input and execute these steps in order, often using a language model for planning and an action agent for execution. Tools and toolkits are essential in these systems; tools interact with the environment according to the agent's goals, while toolkits bundle these tools for specific functionalities, such as interfacing with a SQL database for query execution and table inspection. [28]

F. SIGN LANGUAGE RECOGNITION USING MEDIA PIPE

Mediapipe Framework: MediaPipe Hands facilitates tracking of finger and hand movements by recognizing 21 3D hand features from a single frame through machine learning. The solution enables continuous monitoring on handheld devices and can track multiple hands at once without needing high-end PCs. It aims to provide a well-researched, practical tool while advocating for ethical use and research practices. The system uses a multi-model ML pipeline where a palm detection model first analyzes the entire image to create a hand bounding box. Then, a hand gestures model targets this area to provide precise 3D hand key points, using a method similar to the MediaPipe Facial Mesh solution that combines a landmark model with a face detector. [29]

Vision Based Approach: In Order to use bare hands required a controlled environment with consistent background, lighting, and specialized equipment for real-time analysis. These systems are designed for high accuracy and robustness. The method theoretically mimics how humans interpret environmental data, a challenging concept to implement. Strategies include creating a 3D hand model for comparison with camera images to calculate hand orientation and palm shape, or capturing an image to isolate specific features that inform a partitioning algorithm defining the touch phase. [29]

The development of a sign language recognition system with MediaPipe involves several steps. First, a large dataset of diverse sign language gestures is created and labeled. This dataset is then preprocessed and segmented into training, validation, and testing sets. Hand gestures within the dataset are annotated using MediaPipe's solution to pinpoint hand landmarks and key points. A machine learning model is trained with these annotations to recognize gestures accurately. Once trained, the model is integrated into a MediaPipe pipeline for real-time hand tracking. The system's performance is verified using a separate test dataset to measure its accuracy and efficiency. Finally, the system is deployed to assist users who are deaf or hard of hearing. [29]

Fig. 2. Main points presents on bare hands used to reconstruct a sign. [29]

G. Mixtral of Experts

Mixtral 8x7B, a Sparse Mixture of Experts (SMoE) language model, builds on the Mistral 7B design with 8 feedforward blocks per layer. A router network at each layer selects two experts to process each token, providing access to 47B parameters but only utilizing 13B actively during inference. Trained with a context size of 32k tokens, Mixtral matches or exceeds the performance of Llama 2 70B and GPT-3.5 across all benchmarks, excelling in mathematics, code generation, and multilingual tasks. [33]

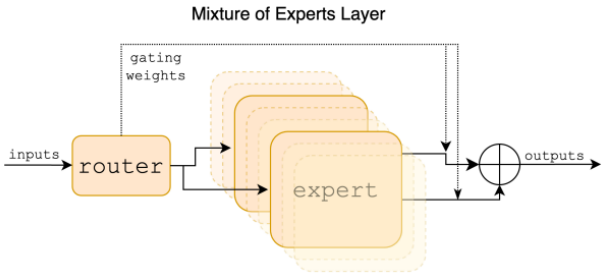

Fig. 3. Mixture of Experts Layer. [33]

Each input vector in Mixtral is assigned to two of the eight experts by a router, with the layer's output being the weighted sum of the outputs from the selected experts. Each expert is a standard feed forward block typical of a vanilla transformer architecture. [33]

Shortly Model Comparison: Mixtral excels in mathematics, code generation, and multilingual tasks, significantly outperforming the Llama 2 70B in these areas. It effectively retrieves information from a context window of 32k tokens, irrespective of sequence length or information location within the sequence. [33]

Mixtral Comparison: Mixtral 8x7B – Instruct, a chat model, has been fine-tuned to follow instructions using supervised fine-tuning and Direct Preference Optimization. This model surpasses GPT-3.5 Turbo, Claude-2.1, Gemini Pro, and Llama 2 70B – chat model in human evaluation benchmarks. [33]

III. Methodology

A. Transcription Made With Whisper:

In order to use whisper, it is necessary to read audio on this formats: mp3, mp4, mpeg, mpga, m4a, wav, and webm. Additionally with a maximum file size of 25 MB (greater than that it's necessary to partition the file). [3]

First of all, it's necessary consider the python library called sound-device, allowing us to obtain a list of all the connected devices, presents in the actual machine (works well with Windows 10 and Ubuntu 22.04 LTS). After that is saved a csv file with all the information about all the possible sound devices that we can use to record audio.

In the following step, it's possible use the previous list (located in a specified folder inside the script), to select the device index that allow us to record audio considering this device. Also in this step (performed by the function: Record_Audio_File_Mic), it's possible to control the duration of the recording, the sample rate of the audio, and where is going to be saved that file. (worth highlighting that the format of file selected was .wav and used the python library scipy.io.wavfile).

After that, it's necessary consider the part of transcription of the audio using whisper, (for that purpose a function called Transcribe_Audio_From_File it's required). In this function we have 3 parameters: The location of the last recorded file, where is going to be saved the transcript audio file in csv format [considering that it's saved like a dataframe], and what version of whisper model is going to be used. (For further mentions save a file recorded with a better mic can allow us to operate with whisper models with lower specs).

B. Usage of Webcam:

To capture the video of a webcam present inside the computer (this part can be changed in the same way that is managed the record of audio from a determined mic), The library opencv or cv2 was used, worth highlighting that also this python library allow us to put text, color dots, modify the color of one dot (for example negative), in the actual frame. This characteristic is exploited to detect the lips movement in company with Media-pipe framework.

C. Detection of lips points:

The way that is detected the movement of lips, is considering a file that have tags representing the corresponding sections, and locations of the points that are predicted to be part of the lips (This file is called ETIQUETAS_LABIOS_03), and it's necessary to run properly the script. After consider the upper and lower lips, the algorithm to calculate the difference is making a sum of all the Y coordinates of all the upper lips points and save that result to a variable, same thing for lower lips points, after that it's necessary consider their difference and save it in another variable, and finally consider their mean or divide by 19 (this is done without knowing the pairs of upper and lower lip points because in some cases the position and rotation of the face introduce a lot of bias).

The next image illustrates the advanced capabilities of the MediaPipe framework, specifically demonstrating its facial landmark detection feature. Here, MediaPipe's Face Mesh model is applied to accurately map the lip points on a subject's face. The highlighted regions on the lips are part of the detailed facial topography created by the model, showcasing how precise and effective MediaPipe is (almost) in real-time facial feature analysis.

Fig. 4. Points detected using predictions based on MediaPipe. (Also used this resource to print important messages instead the python terminal)

D. Using Langchain, Ollama & Llama3 :

In order to use Langchain and Ollama it's necessary install their dependencies. (In the first run after installing all the corresponding dependencies we can use the ollama setup instantly, but in Windows 10 it's necessary execute another command to start the server, after that we can interact with the installed models or install new ones).

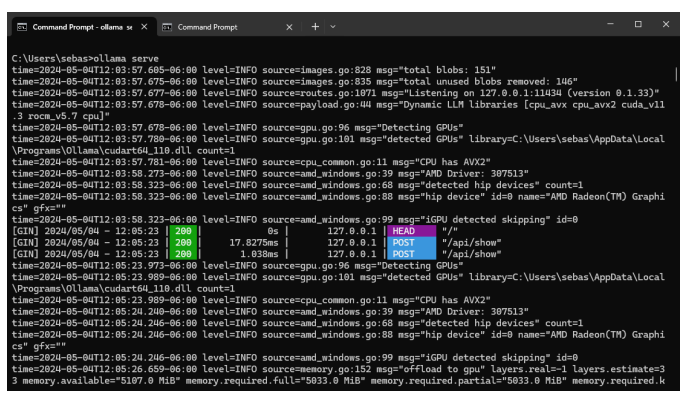

Fig. 5. Server locally mounted of ollama turned on. All the information about the usage of hardware displayed

To use properly ollama inside python script it's necessary consider the next command: "Ollama(base_url='http://localhost:11434', model='Llama3')", the base url it's a parameter that allow us to run the LLM locally, and the parameter model tell us what model are we going to instanciate inside an object.

E. Separate tests:

For purposes of the project, become necessary identify how much hardware was required to use the LLM's locally (and how it's done), for this case the application of LM Studio was used, in this app we can "chat" with the LLM's in a few steps, the only requirement it's download the file that contains the LLM and verify the hardware recommended to run the Model. (In this part was probe the Mixtral variants, Hermes, Wizardcode, Zephyr 3B, Starling, LLama2, LLama3, performing best results of response time with LLama 2 and their successor LLama3).

F. Control the movement of Dron inside GAZEVO:

The Practice 01 PID program uses two CSV files: one recording the drone's position through measurements from the topic /simple_drone/gt_pose, and another for manually entered desired positions at /DESIRED_POSITION/DESIRED_POSITION.csv. The current position data is averaged, and desired positions are manually set for simulations. The Python Glob library dynamically manages file paths to process only one relevant CSV per folder automatically. An additional folder holds important data such as drone position, adjustments, and errors. A function then creates five graphs showing these data across iterations for each axis.

Position Control: Position control of the drone utilizes methods from the Keyboard package to create a new class with modified methods, allowing movement at specified speeds and directions across three axes.

Chosen Controller Type: A PID controller was chosen, with manual tuning employed to determine the coefficients. The tuning process involved adjusting the proportional constant (Kp) to identify oscillation points and setting it to a value that minimized abrupt oscillations, while the integral (Ki) and derivative (Kd) constants were fine-tuned to optimize performance.

Constant Tuning: The final configuration yielding the best results involved setting the sampling time (dt) to 0.01 seconds, with Kp set at 1, Ki at 0.0001, and Kd at 0.01, under a maximum speed of 0.3. This setup allowed the drone to reach the desired position efficiently within 30 iterations, demonstrating the effectiveness of the chosen PID parameters in managing the drone's movement in Gazebo.

This program was modified to catch the orientation, and adapted to use in order with langchain, whisper, ollama and LLama 3.

IV. Experimental Framework

To capture video it is necessary to consider not having the following biases.

- Viewing angle to the camera.
- Separation distance from face to camera.
- Strong body movements.
- Strong movements of the face or head.
- Covering the lips, the media-pipe system makes the prediction but is not able to detect movement with covered lips.
- Environment with too much or too little light.
- Ram memory saturation affects the performance of the camera.

Interruption of the video frame by whisper: In the time that Whisper it's triggered, causes an interruption to the OpenCV in the shown video, this alteration ends at the time when whisper finalize of processing and transcribing the audio recorded (on average 40 - 70 seconds depending on the state of the computer used). In this phase was probed Multi-threading, Multiprocessing, and Concurrent. Futures. The last one, offers better results in the way of concurrent whiles, but due to the actual limitations of whisper, and the CODEC selected, the concurrency cannot be applied beyond the limits to consider ASR (Automatic Speech Recognition) in real time (principally due to the limitations of the hardware). 1 minute in average only for processing text with whisper.

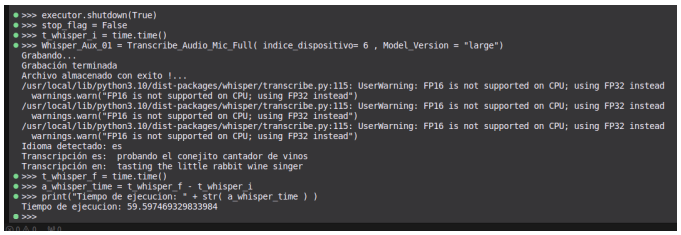

Fig. 6. Example of the elapsed time on the execution of whisper, it takes more with the implementation of concurrent.futures.

Why LLama3 takes to long ? This is mainly due to the fact that the considered instance of LLama 3 runs locally considering the resources present in the PC hardware, so the delay incurred will be significantly greater than considering a model that exists in the cloud (such as ChatGPT), although good results can be obtained to get the corresponding classifications or labeling thanks to the LLM. (Same justification it's for LLama 2).

Number of sentence considered in the comparison graphs between LLAMA 2 and LLAMA3 .

- **1 -** "I want to go to a position".
- **2 -** "Move the dron 2 meters to the right".
- **3 -** "what it's the actual position of the Dron".
- **4 -** "How far it's the dron from the origin?".
- **5 -** "How fast the dron it moving?".

- **6 -** ”How fast the dron it’s rotating?”.
- **7 -** ”How many revolutions per second it’s performing the dron”.
- **8 -** ”I want to put a tag in this point”.
- **9 -** ”Recover x,y,z”.
- **10 -** ”Recover Vx,Vy,Vz”.
- **11 -** ”Recover Wx,Wy,Wz”.
- **12 -** ”Where is facing the Dron?”.
- **13 -** ”Land please”.
- **14 -** ”Take off the Dron please”.
- **15 -** ”Can you stop Dron please?”.
- **16 -** ”I don’t going to use the dron know, can you please stop the dron?”.
- **17 -** ”If the Dron is stopped, please land the dron”.
- **18 -** ”If the Dron has Movement, please stop the dron”.
- **19 -** ”Is the dron moving right now?”.
- **20 -** ”The Dron it’s rotating?”.

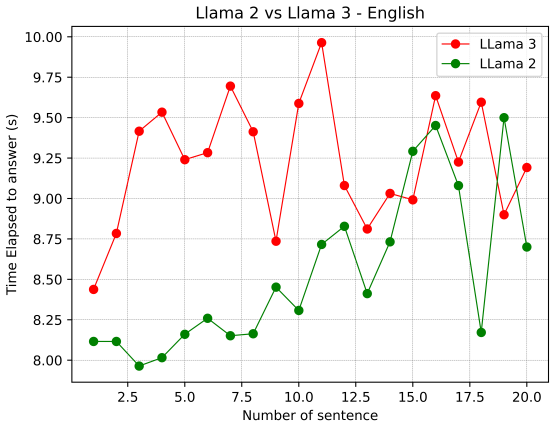

Fig. 7. Average time of executing llama3 and llama2 inside the computer locally, offline, considering the sentences in English. (lower it’s better).

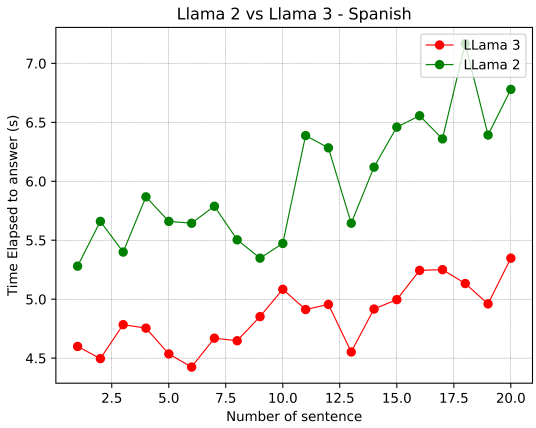

Fig. 8. Average time of executing llama3 and llama2 inside the computer locally, offline, considering the sentences in Spanish. (lower it’s better).

V. Conclusions

It’s possible to use Whisper combined with langchain and ollama to make it easier the installation and the execution inside the code, and all the anterior resources combined with the framework media-pipe that also use ML to predict the desired points of the face or body parts, it’s possible to emulate a lot of technologies, for example a digital assistant in only code and enhanced with the LLM of our election, Translation in almost real time, make queries of tables without even need SQL or write the context token of the querie, making useless in some cases SQL or even the written Natural Language it’s not necessary. Another context of use of this project due to his versatility, it’s the scenario of therapy that the only way to know how a patient feels or what’s to talk without use a keyboard or pencil to understand their words.

Improvements of the Project or Future Work:

- Generate more clasifications about input prompts detecte by whisper.
- Try to use more instances of whisper with multiprocessing or adapt with multi-threads, to eliminate the bias due to the noise of environment.
- Improve the movement of the dron in the simulation to consider using quaternions.
- Try to instantiate more than only 1 LLM in the same machine.
- Improve the performance of the use of Whisper and LLM or consider a lower number of tokens.
- Eliminate the background using Mediapipe or another python library.
- Use CUDA with Whisper & the LLM’s to perform better times of response.
- Improve the audio recording with reducing the noise as far as be possible inside python.
- Use Movie Py to improve the visualization of the web cam video cleaning all the images.
- Using the framework Deepspeed and consider make text to probe how much time is saved for use this resource.
- Improve the PID controller to afect position, orientation and rotation.
- Use Heavier llms in a poweful machine to try to tag the outputs processed by whisper.
- Try to use a better quality camera.

References

[1] OpenAI, ” *Introducing Whisper* ”, Retrived 04/05/2024
[2] OpenAI, ” *Whisper - Installation* ”, Retrived 04/05/2024
[3] Ramsri Goutham Golla, ” *Here Are Six Practical Use Cases for the New Whisper API* ”, 06/03/2023
[4] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever, ” *Robust Speech Recognition via Large-Scale Weak Supervision* ”
[5] MediaPipe Team, ” *Face landmark detection guide* ”, Retrived 04/05/2024
[6] Chuoling, ” *ML solutions in MediaPipe* ” , Retrived 04/05/2024
[7] Google - Mediapipe Dev Team ” *MediaPipe* ”, Retrived 04/05/2024
[8] Kukil , Pranav Durai, ” *What is MediaPipe?* ” , Retrived 04/05/2024
[9] LangChain - Team, ” *LangChain* ”, Retrived 04/05/2024
[10] LangChain - Team, ” *LangChain - Documentation* ”, Retrived 04/05/2024

[11] LangChain - Team, *" PipelineAI "*, Retrived 04/05/2024
[12] Tech With Tim , *" LangChain Explained In 15 Minutes - A MUST Learn For Python Programmers "*, 13/12/2023
[13] Matthew Berman , *" OpenDevin Tutorial (Open-Source Devin) - Build Entire Apps From a Single Prompt "*, 30/03/2024
[14] AI Jason, *" Hugging Face + Langchain in 5 mins — Access 200k+ FREE AI models for your AI apps "* , 10/06/2024
[15] Matthew Berman, *" New $10m Open-Source Foundational LLM Is AMAZING! (DBRX by Databricks) "*, 02/04/2024
[16] Matt Williams, *" Using Langchain with Ollama and Python "*, 11/08/2023
[17] Isaac Chung ,*" What is Ollama? A shallow dive into running LLMs locally "*, 07/01/2024
[18] Meta DeV Team, *" Ollama "*, Retrived 04/05/2024
[19] Ollama DeV Team, *" Ollama Models "*, Retrived 04/05/2024
[20] Meta LLama Dev Team, *" Build the future of AI with Meta Llama 3 "*, Retrived 04/05/2024
[21] Meta LLama Dev Team, *" Meta Llama 3 "*, Retrived 04/05/2024
[22] Philipp Schmid et. al. Huggin Face, *" Welcome Llama 3 - Meta's new open LLM "*, 18/04/2024
[23] Meta-LLama Dev Team *" Introducing Meta Llama 3: The most capable openly available LLM to date "*, 18/04/2024
[24] Andrés A. Ramírez-Duque, Mary Ellen Foster, *" A Whisper ROS Wrapper to Enable Automatic Speech Recognition in Embedded Systems "*, 13/02/2023
[25] Vijayshri Khedkar et. al.,*" Web Scraping for Ovarian Cancer Detection: Utilizing Open-Source WhisperAI for Identifying Relevant Terminology and Improving Early Diagnosis "*, 07/08/2023
[26] Keivalya Pandya, Dr. Mehfuza Holia, *" Automating Customer Service using LangChain "*, 30/12/2023
[27] Eduardo Nascimento, et. al., *" A Family of Natural Language Interfaces for Databases based on ChatGPT and LangChain "*, Retrived 04/05/2024
[28] Oguzhan Topsakal and Tahir Cetin Akinci, *" Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast "*, 12/07/2023
[29] Ranjitha R Shetty and Saritha Shetty, *" SIGN LANGUAGE RECOGNITION USING MEDIA PIPE "*, 01/05/2023
[30] Zhengliang Liu et. al., *" Radiology-Llama2: Best-in-Class Large Language Model for Radiology "*, 29/08/2023
[31] Bohdan M. Pavlyshenko, *" Financial News Analytics Using Fine-Tuned Llama 2 GPT Model "*, 29/09/2023
[32] Pranav Gade, Simon Lermen, Charlie Rogers-Smith, Jeffrey Ladish, *" BadLlama: cheaply removing safety fine-tuning from Llama 2-Chat 13B "*, 21/03/2024
[33] Albert Q. Jiang et. al.,*" Mixtral of Experts "*, 08/01/2024
[34] ChatGPT, *" ChatGPT "* Retrived 05/05/2024